Departamento de Ingeniería Eléctrica
Facultad de Ingeniería
Universidad de Concepción
Concepción, Chile.

549253-1
Taller de aplicación TIC 1

Mini proyecto 2

TIC is Among Us v2.0

Profesor: Vincenzo Caro Fuentes

Integrantes: Javier Espinoza Arcos
Ignacio Aguilar Contreras
Sebastián Tecay Hermosilla

Concepción, octubre de 2026

Índice

1. Actividad 1: TIC is Among Us v2.0 **2**

1.1. Esquema de conexiones del circuito 2
1.2. Conexiones y función de los componentes 3
1.3. Diagrama de bloques del hardware 3
1.4. Diagrama de bloques del software y la comunicación 4
1.5. Descripción de los minijuegos 4
1.5.1. GameID 1: Caja fuerte 4
1.5.2. GameID 2: Calibrar la nave 5
1.5.3. Duración y entrega de resultados 5
1.6. Comunicación con el host y registro en el Lobby 6
1.7. Rondas 1 y 2: asignaciones y sabotajes 6
1.8. Ronda final y recuperación de vida 7
1.9. Interfaz gráfica y función de sus elementos 8
1.10. Elementos de presentación incorporados 8
1.10.1. Vista del minijuego Caja fuerte 9
1.10.2. Vista del minijuego Calibrar la nave 10
1.10.3. Vista de la ronda final 11
1.11. Pruebas y resultados 12
1.12. Problemas encontrados y soluciones 12

Referencias y material utilizado **12**

1. Actividad 1: TIC is Among Us v2.0

El proyecto consiste en juego desarrollado en Python para una Raspberry Pi. El jugador utiliza un encoder, un sensor de inclinación y un sensor ultrasónico para completar dos minijuegos: *Caja fuerte* y *Calibrar la nave*. La interfaz gráfica presenta las instrucciones, las lecturas y el progreso, mientras que un buzzer entrega una respuesta sonora al ganar o perder.

La Raspberry Pi se comunica con un host central, encargado de coordinar las etapas y mantener el estado oficial de la partida. El cliente recibe asignaciones, ejecuta las acciones del jugador e informa los resultados. En las secciones siguientes se describe el funcionamiento implementado y los resultados de las pruebas realizadas por el grupo.

1.1. Esquema de conexiones del circuito

La Fig. 1 presenta el montaje de referencia entregado para el informe. Incluye la Raspberry Pi, el encoder KY-040, el MPU6050, el sensor HC-SR04, el buzzer KY-006.

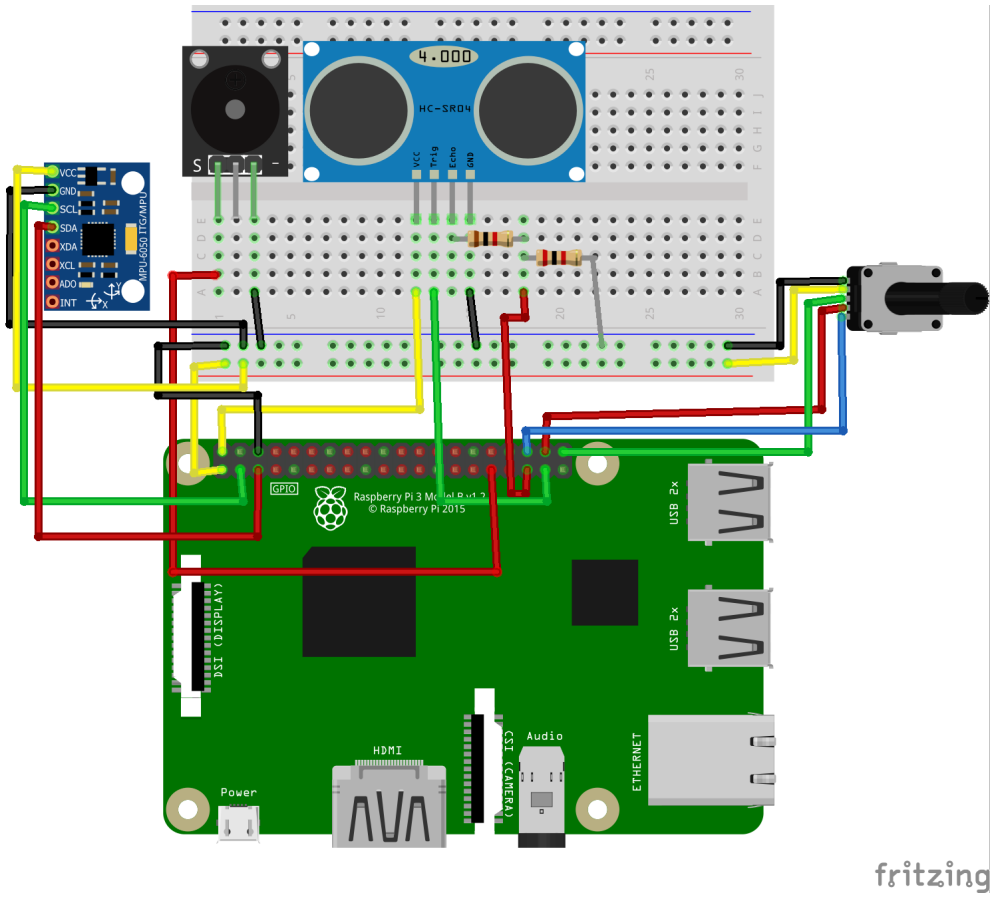


Fig. 1: Representación del montaje en protoboard aportada para el proyecto.

1.2. Conexiones y función de los componentes

La Tabla 1 resume las conexiones indicadas en el diagrama de bloques y confirmadas en las constantes de `final.py`. Los números corresponden a GPIO, no a la posición física de los pines del conector.

Tabla 1: Asignación de señales según la información del proyecto.

Componente	Señal	GPIO	Función
Encoder KY-040	CLK / DT	16 / 20	Giro para seleccionar números.
Encoder KY-040	SW	21	Confirmación y pulsaciones.
MPU6050	SDA / SCL	2 / 3	Comunicación I^2C para leer la inclinación.
HC-SR04	TRIG	26	Disparo de la medición de distancia.
HC-SR04	ECHO	19	Recepción a través del divisor resistivo.
Buzzer KY-006	S	6	Señal PWM para efectos sonoros.

1.3. Diagrama de bloques del hardware

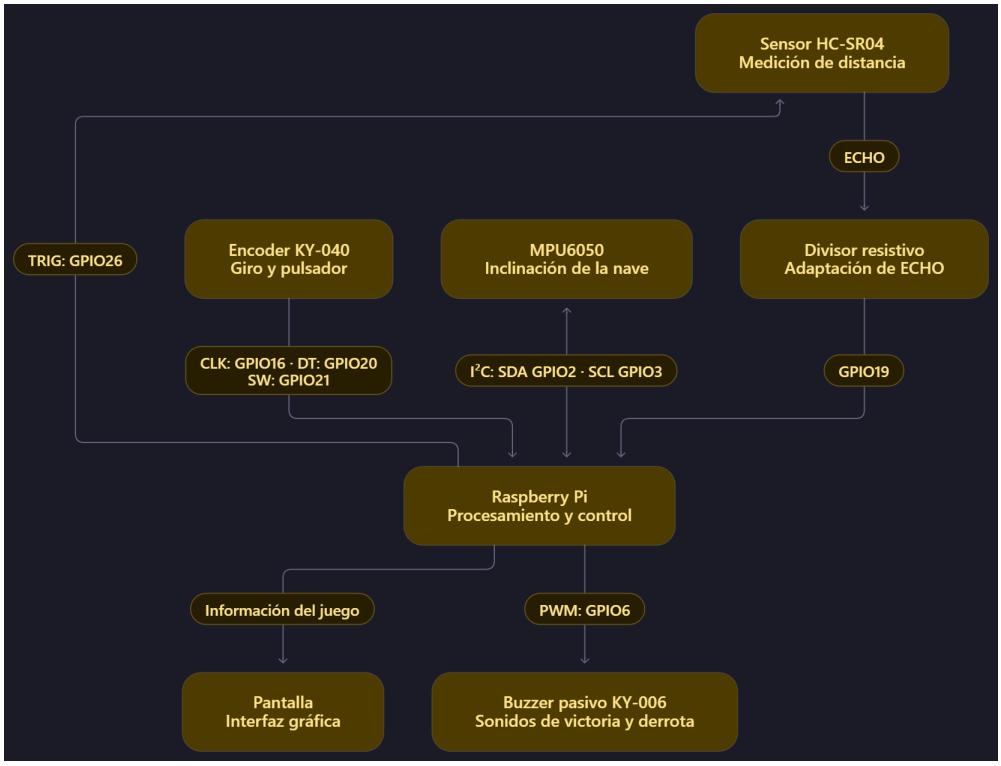


Fig. 2: Entradas, procesamiento y salidas del sistema, según el diagrama aportado.

1.4. Diagrama de bloques del software y la comunicación

El programa recibe las instrucciones del host, lee las entradas físicas y evalúa el cumplimiento de las misiones. Los resultados y las solicitudes de recuperación se envían al host para su confirmación. La interfaz muestra los objetivos, las mediciones y las vidas recibidas, mientras que el buzzer reproduce los sonidos asociados al resultado.

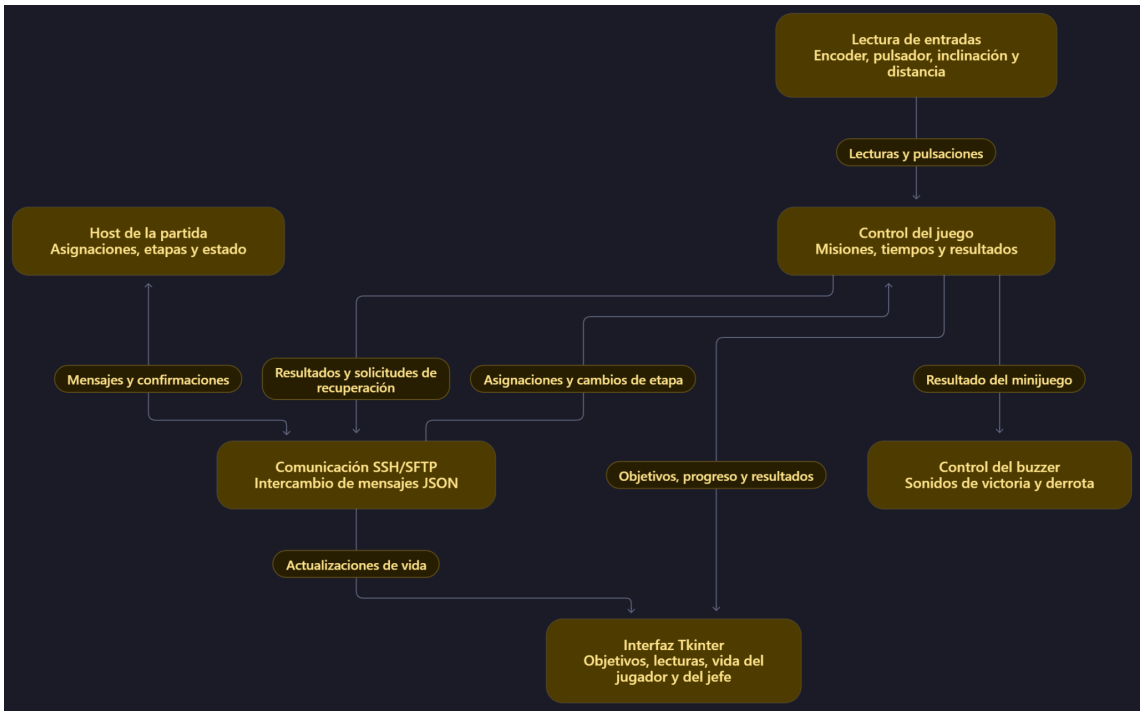


Fig. 3: Relación entre la lectura de sensores, el control del juego, la interfaz y la comunicación SSH/SFTP.

La separación mostrada en la Fig. 3 permite describir la función de cada bloque. La lectura de entradas entrega los valores necesarios para jugar; la lógica determina el avance local; y la comunicación informa al host de las acciones realizadas. El host conserva la autoridad sobre el estado global de la partida.

1.5. Descripción de los minijuegos

1.5.1. GameID 1: Caja fuerte

El programa genera tres números aleatorios entre -8 y 8, excluyendo el cero. El jugador gira el encoder y presiona su eje para confirmar cada número. Si acierta, el conteo del encoder vuelve a cero y continúa con el siguiente objetivo; si confirma un valor incorrecto, termina con `Lose` y puntaje 0. Al confirmar los tres números obtiene `Win` y puntaje 100.

En el ejemplo ilustrativo, la combinación es $[4, -6, 2]$. El primer número ya fue confirmado y el jugador debe llegar a -6. Una confirmación incorrecta activa la alarma indicada en las instrucciones. La combinación mostrada es un ejemplo, no una medición ni un resultado de prueba.

1.5.2. GameID 2: Calibrar la nave

Este minijuego tiene dos fases y comparte un límite de 15 segundos entre ambas. Primero se elige un ángulo aleatorio de magnitud entre 20° y 40°, con signo positivo o negativo. El jugador debe mantenerlo durante 2 segundos con una tolerancia de ±7°.

Después se solicita una distancia aleatoria de 10 a 30 cm, con tolerancia de ±3 cm, durante otros 2 segundos. Si la lectura sale del intervalo permitido, el tiempo de permanencia de esa fase vuelve a cero. Completar ambas fases entrega `Win` y puntaje 100.

Durante la primera fase, una inclinación de magnitud superior a 70° produce derrota. Durante la segunda, una distancia inferior a 3 cm produce derrota. Cada condición se comprueba solo en su fase correspondiente. El ángulo de 30° y la distancia de 20 cm mostrados en el mockup son ejemplos compatibles con estos rangos.

1.5.3. Duración y entrega de resultados

La constante `TIEMPO_MAX` fija 15 segundos para el juego activo. Las funciones devuelven `Result` y `Score`; después se reproduce la melodía, de modo que el tiempo total hasta informar el resultado también incluye esa reproducción. Las reglas base son:

Tabla 2: Resultados exigidos antes de aplicar sabotajes.

Result	Score	Condición
`Win`	100	Completar correctamente el objetivo.
`Lose`	0	Fallar el objetivo del minijuego.
`TimeOut`	1 a 99	Agotar el tiempo; el puntaje depende del avance.

Al agotarse el tiempo, `score_timeout` redondea el porcentaje de avance y lo limita entre 1 y 99. En Caja fuerte, el avance combina los números acertados y la cercanía al objetivo actual. En Calibrar la nave considera las fases completadas y el tiempo mantenido dentro de la tolerancia de la fase actual.

Los botones de prueba ejecutan los minijuegos sin conectarse al host. El resultado y el puntaje se muestran en la interfaz y se imprimen en consola mediante `gui_log`. El código incluye bloques `finally` para cerrar los recursos de lectura utilizados durante el juego.

1.6. Comunicación con el host y registro en el Lobby

El cliente utiliza Paramiko para abrir una conexión SSH y una sesión SFTP. Toda comunicación es iniciada por el jugador.

Para registrarse, el cliente envía `Join` y espera `Accepted`, verificando la coincidencia entre `ReplyTo` y `EventID`. El contador de eventos se conserva en un archivo local entre ejecuciones. Los mensajes se suben con extensión temporal y luego se renombran a `.json`. Se incluyen intentos de reconexión.

1.7. Rondas 1 y 2: asignaciones y sabotajes

Antes de comenzar una ronda, el host envía `StageStart`. El cliente prepara la etapa y responde `StageReady`; luego verifica el ACK `StageReadyAccepted`. Al recibir `StageActive`, la interfaz anuncia la ronda activa. Sin embargo, la rama que atiende `Assign` no comprueba explícitamente que la activación haya ocurrido; el flujo presupone que el host envía los eventos en el orden definido.

Durante las rondas, `Assign` indica el minijuego mediante `GameID` y el intento mediante `Attempt`. Al terminar, el cliente envía `Result`, conservando ambos valores e incluyendo el resultado y el puntaje. Debe esperar `ResultAccepted` y comprobar su `ReplyTo` antes de cerrar el intercambio.

Los mensajes `Sabotage` se acumulan en una lista que se vacía después de la siguiente asignación. Si llegan varios, se suman las reducciones de tiempo y de puntaje; el tiempo disponible queda limitado a un mínimo de 1 segundo. La pauta reserva estos mensajes para la Ronda 2:

Tabla 3: Efectos de sabotaje definidos en el enunciado.

Effect	Value	Comportamiento esperado
`Delay`	2 a 5	Reducir el tiempo disponible en esa cantidad de segundos.
`Disable`	1	Omitir el siguiente juego y enviar `Lose` con puntaje 0.
`ScoreSteal`	10 a 30	Restar el valor al puntaje antes de enviarlo, manteniéndolo entre 0 y 100.

La implementación contiene los tres efectos. `ScoreSteal` modifica el puntaje después del juego y conserva el resultado original; por ejemplo, una victoria con robo de 20 puntos se informa como `Win` y puntaje 80. Durante las pruebas realizadas por el grupo, los sabotajes funcionaron correctamente.

1.8. Ronda final y recuperación de vida

La ronda final utiliza el pulsador del encoder. El host comunica el inicio mediante `FinalStart`, cuyo campo `ReactionPresses` indica cuántas pulsaciones completan un ciclo. Después de preparar la etapa y realizar el intercambio de confirmación, el cliente espera `StageActive`.

Al completar el ciclo se envía `Recover` con un nuevo `EventID`. La función espera `RecoverAccepted` y comprueba `ReplyTo`. Solo incrementa las recuperaciones confirmadas si recibió el ACK. Sin embargo, reinicia el contador de pulsaciones incluso cuando la espera termina sin confirmación. Esto requiere ajuste para garantizar la regla de la pauta: no iniciar un nuevo intercambio mientras el anterior siga sin confirmar. El valor de pulsaciones se lee de `ReactionPresses`, con 10 como valor inicial de respaldo.

El cliente debe continuar consultando mensajes durante esta etapa. Si recibe `Eliminated`, deja de enviar recuperaciones y espera el cierre global. Al recibir `GameOver`, termina la partida: según la pauta, el jugador sobrevive si no había sido eliminado; si ya había recibido `Eliminated`, finaliza como derrotado. Las vidas y el estado global los determina el host.

1.9. Interfaz gráfica y función de sus elementos

La interfaz descrita utiliza Tkinter. Mantiene una estructura común para los dos minijuegos y la ronda final, cambiando las instrucciones, la lectura principal y el progreso según la actividad. La Tabla 4 resume los widgets identificados en `crear_ventana()` del código revisado.

Tabla 4: Widgets y función en la interfaz descrita.

Elemento	Función
`Label`: `lbl_etapa`	Identifica al jugador y la etapa activa.
`Label` y `ttk.Progressbar`	Presentan la vida del jugador y del jefe recibida desde el host.
`lbl_titulo` y `lbl_instruccion`	Muestran el nombre del juego y la acción requerida.
`lbl_objetivo`	Destaca la misión en azul y negrita.
`lbl_grande`	Presenta el número del encoder, el ángulo, la distancia o el conteo de pulsaciones.
Etiquetas auxiliares	`lbl_detalle`, `lbl_tiempo`, `lbl_avance` y `lbl_resultado` muestran el progreso, el tiempo y el resultado.
`Button`	Permite probar cada minijuego o iniciar la partida conectada.
`Text`: `caja_log`	Muestra eventos, mensajes, confirmaciones, resultados y errores.

El tiempo y el avance se presentan como texto. Las barras de progreso se utilizan para las vidas. En los estados en curso ilustrados, la etiqueta de resultado permanece vacía.

Las figuras siguientes son capturas de ejecución real. Las vidas, los tiempos, las lecturas y los mensajes son ejemplos simulados.

1.10. Elementos de presentación incorporados

En el código revisado, la interfaz incluye una misión destacada, una lectura principal de gran tamaño, indicadores de vida, botones de prueba local y un registro de eventos. Estos elementos permiten reconocer la acción requerida y seguir el progreso del jugador. El buzzer complementa la información visual con sonidos de victoria y derrota.

1.10.1. Vista del minijuego Caja fuerte

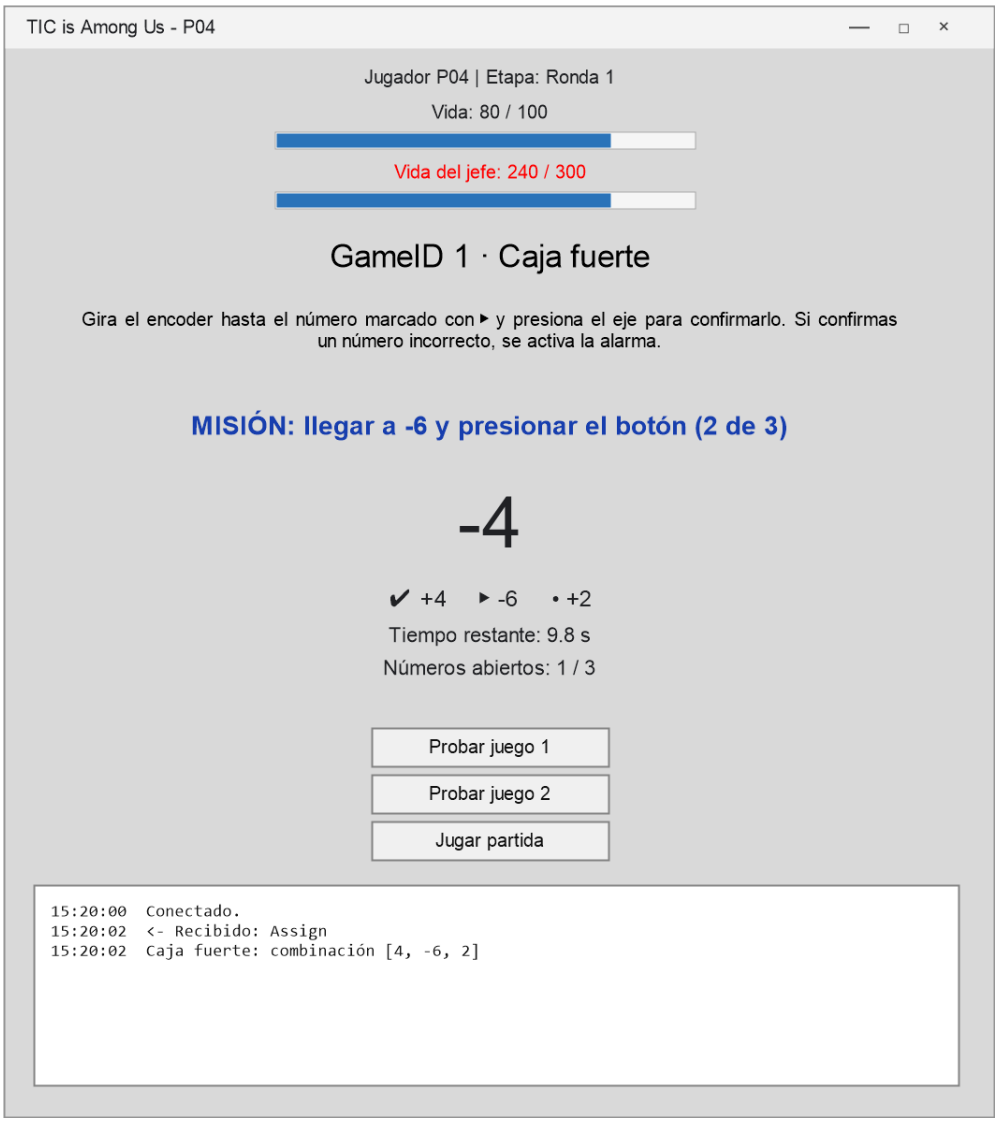


Fig. 4: Caja fuerte. Se muestran las instrucciones, el objetivo, la lectura del encoder, las vidas, los botones y el registro de eventos. Valores ilustrativos.

En la Fig. 4, el encoder marca -4 y el objetivo actual es -6. La pantalla indica que se ha completado uno de los tres números. La combinación y el tiempo restante ayudan a reconocer cuánto falta para terminar la misión.

1.10.2. Vista del minijuego Calibrar la nave

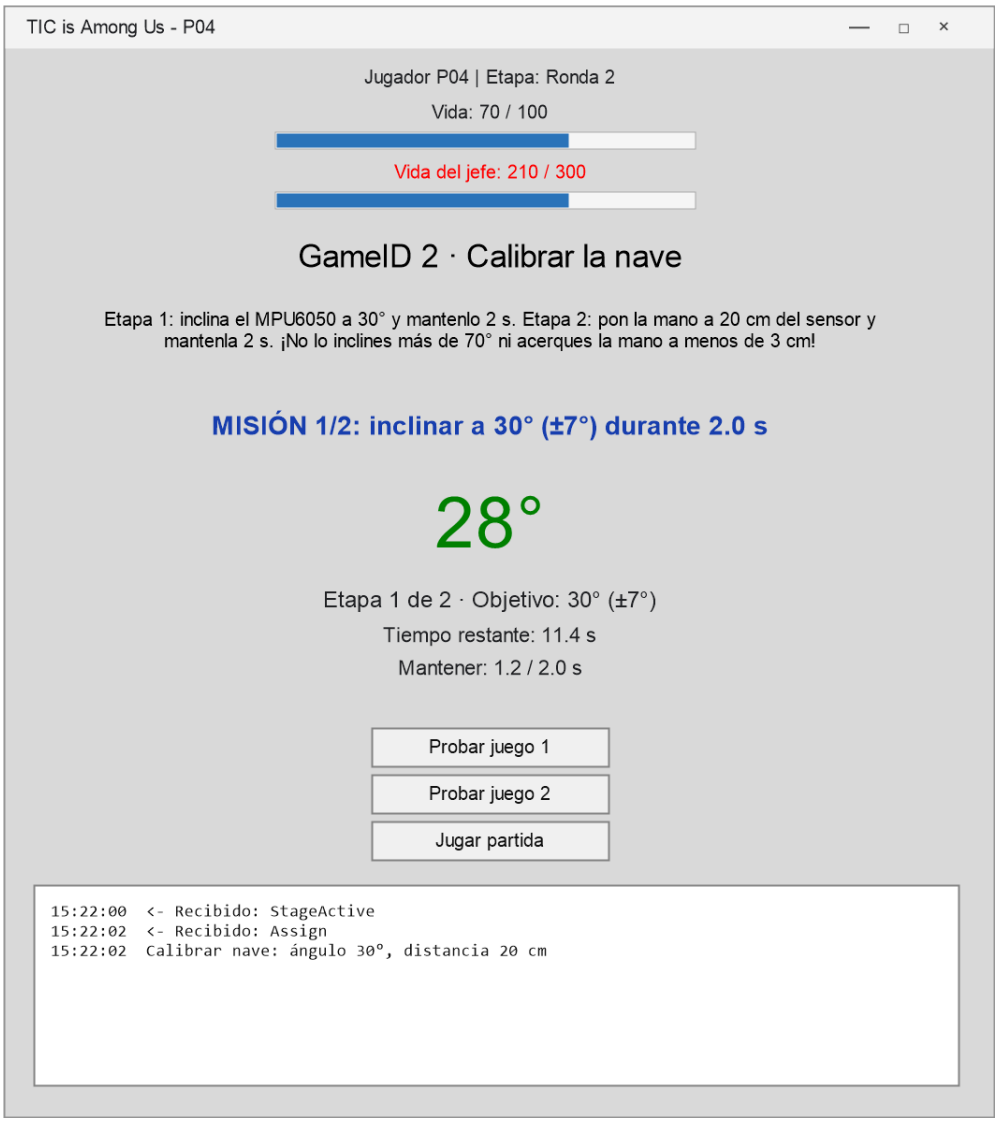


Fig. 5: Calibrar la nave durante la fase de inclinación. La fase de distancia reutiliza los mismos elementos de lectura y progreso. Valores ilustrativos.

En la Fig. 5, la lectura es de 28° para un objetivo de $30° \pm 7°$. El ejemplo muestra 1,2 de los 2 segundos de permanencia requeridos y 11,4 segundos restantes. Estos números explican la presentación visual, pero no demuestran la precisión del sensor ni el funcionamiento del montaje.

1.10.3. Vista de la ronda final

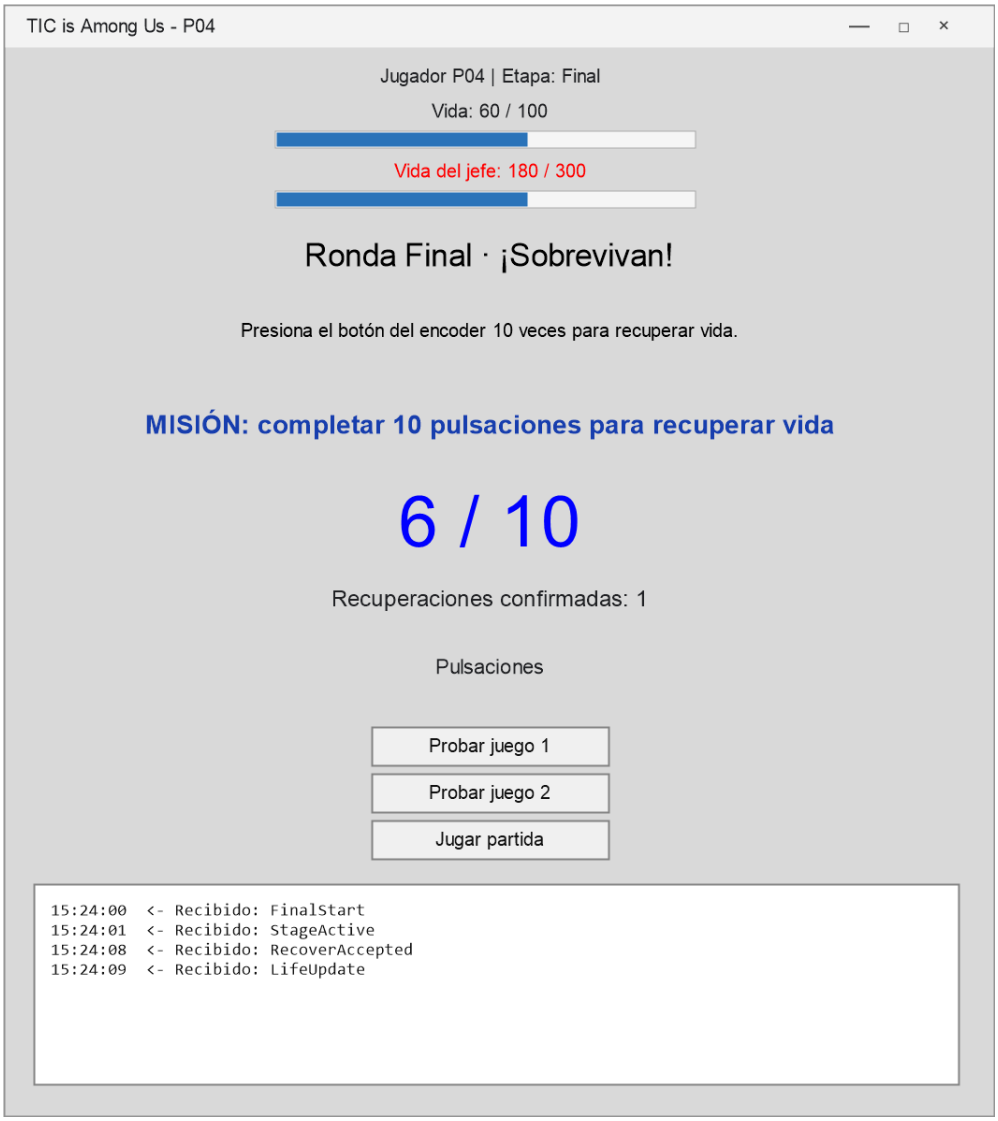


Fig. 6: Ronda final. Se presenta el conteo de pulsaciones y la cantidad de recuperaciones confirmadas por el host. Valores ilustrativos.

En la Fig. 6, el jugador lleva 6 de las 10 pulsaciones del ciclo y aparece una recuperación confirmada. La pantalla también presenta la vida del jugador y la del jefe. La recuperación debe distinguirse del conteo local: completar las pulsaciones genera una solicitud y el host confirma su procesamiento.

1.11. Pruebas y resultados

Se probaron los dos minijuegos y la partida conectada al host. Ambos minijuegos funcionaron correctamente, al igual que las rondas, los sabotajes y la recuperación de vida en la ronda final.

El video de las pruebas realizadas se encuentra disponible en el siguiente enlace:

https://www.youtube.com/watch?v=EdffZihCuW8

Tabla 5: Resultados de las pruebas de funcionamiento.

Prueba	**Resultado**
Caja fuerte	Funcionó correctamente.
Calibrar la nave	Funcionó correctamente.
Conexión al host	Se logró conectar después de varios reintentos.
Rondas 1 y 2	Funcionaron correctamente una vez establecida la conexión.
Sabotajes	Funcionaron correctamente.
Ronda final y recuperación	Funcionaron correctamente.

1.12. Problemas encontrados y soluciones

La principal dificultad fue establecer la conexión con el host, ya que no se logró en los primeros intentos. Para resolverlo, se volvió a intentar la conexión varias veces hasta que fue posible ingresar a la partida. Una vez conectados, los minijuegos y las demás etapas funcionaron correctamente. No se identificó una causa específica para la dificultad de conexión.

Referencias

1. Universidad de Concepción, Departamento de Ingeniería Eléctrica. *MiniProyecto 2, Taller de Aplicación TIC I.* Profesor Vincenzo Caro, 5 de octubre de 2026. Actividad 1, ítems 1.1 a 1.4 y Anexo A. Documento entregado con el proyecto.